

Estruturas de dados Indexada

Vetor e Lista

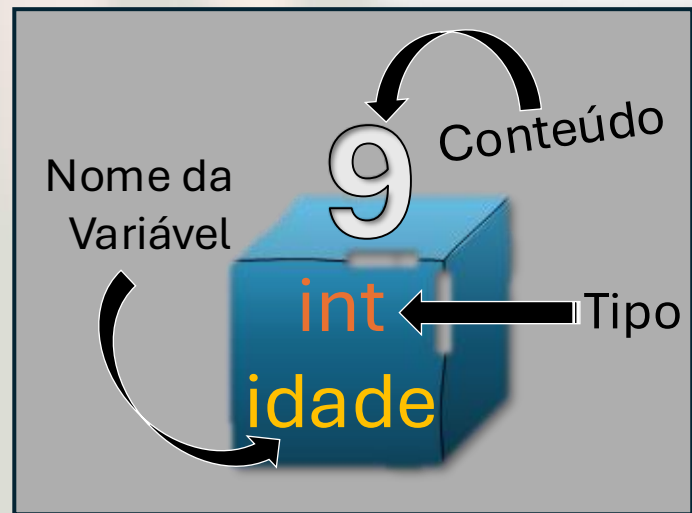
Prof. Edson de Oliveira

Instagram | Tiktok: [**@BeabaProgramacao**](#)

Youtube: [**@BeabaDaProgramacaoEds**](#)

Variáveis de memória

Antes de iniciarmos este novo conceito, vamos nos lembrar de como uma variável de memória comum funciona.



Toda variável de memória tem:

- Um nome
- Um tipo e
- Armazena um conteúdo por vez

Variáveis indexadas

É possível guardar várias informações em uma mesma variável?

Sim, por meio das variáveis indexadas, como listas, vetores ou matrizes.

O que é uma variável indexada?

É uma variável cujos conteúdos são armazenados em elementos referenciados por índices, ou posições, diferentes dentro da mesma estrutura.

O que é um índice?

São números inteiros que começam em 0 (zero) e vão até o tamanho da estrutura menos um. Eles representam a posição de cada elemento armazenado, de forma semelhante às células de uma planilha.

Antes de prosseguirmos, precisamos saber:

Python não trabalha com conceito de Vetor, apenas Lista!

E agora?

E Python conseguimos simular um vetor.

Como assim?

Basta criar uma lista e nela inserir dados do mesmo tipo, parecendo um vetor.

Então qual a diferença de uma Lista para um vetor?

Vetor: Tamanho pré-definido.

Lista: Tamanho dinâmico.

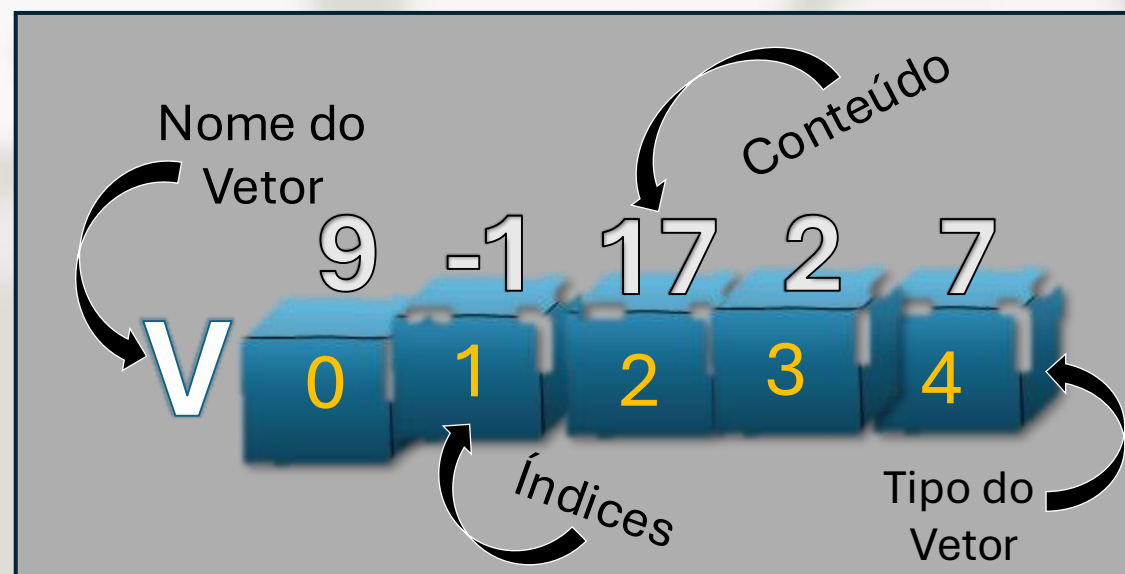
Vetor: elementos homogêneos.

Lista: elementos heterogêneos.

Variáveis indexadas

Do tipo Vetor

Todo vetor tem um nome, índices, elementos (conteúdo) e um tipo.

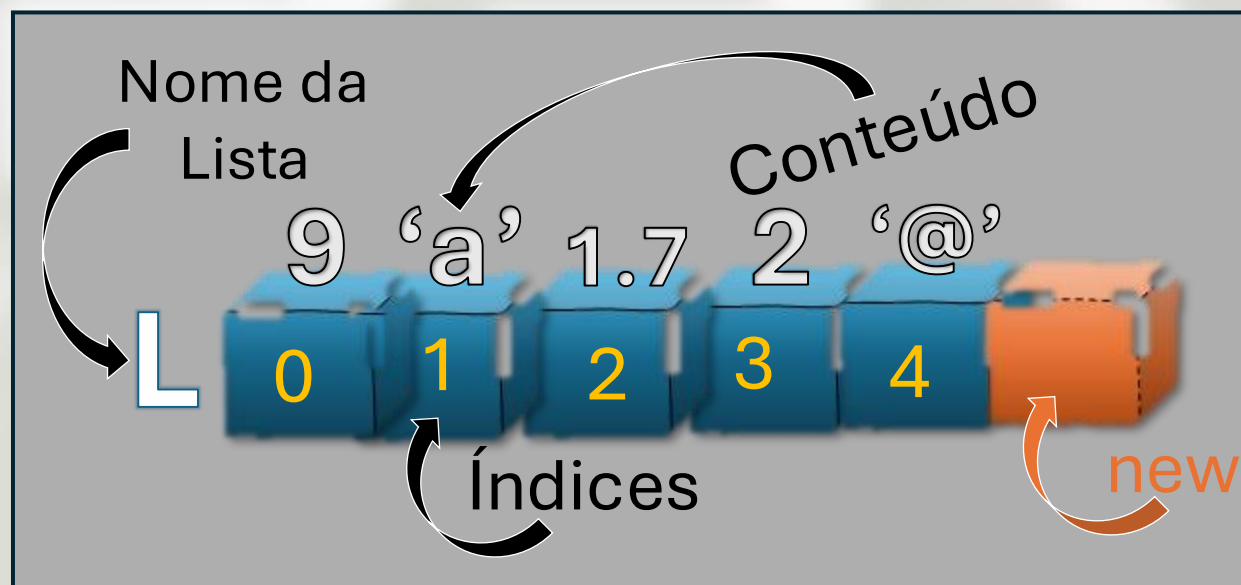


- Todos os elementos (conteúdo) são do mesmo tipo
- O tamanho do vetor é fixo e definido pelo programador

Variáveis indexadas

Do tipo Lista

Todo lista tem um nome, índices, elementos (conteúdos) e um tipo.



- Os elementos (conteúdo) podem ser de tipo diferente
- O tamanho da lista é dinâmico, ou seja, pode aumentar ou diminuir

Manuseando um “vetor” em Python:

```
# Atribuindo valores em um 'vetor'
# 0 1 2 3 4 <= Índices
vetor = [15, -20, 30, 23, 14]
# Exibindo o vetor
print(vetor)
>> [15, -20, 30, 23, 14]
# Exibindo a terceira posição do vetor (índice 2)
print(vetor[2])
>> 30
# Modificando a primeira posição (índice 0)
vetor[0] = 22
# Exibindo o vetor
print(vetor)
>> [22, -20, 30, 23, 14]
```

Legenda:

Amarelo: output da linha anterior

Demais cores: linhas de código

Exercícios com “vetor” em Python:

v	0	1	2	3	4
	45	-89	32	-12	33

Considere o vetor de nome ‘v’ acima e faça as seguintes rotinas:

1. Exibir somente os números negativos.

```
>> -89 -12
```

2. Somar os elementos do vetor.

```
>> 9
```

3. Exibir os números ímpares contidos no vetor.

```
>> 45 -89 33
```

4. Exibir os elementos ímpares.

```
>> -89 -12
```

5. Verificar se um elemento dado pelo usuário existe no vetor.

```
x = 32
```

```
>> 0 conteúdo de 32 existe no vetor.
```


Manuseando uma Lista em Python:

```
# Atribuindo valores em uma lista
# 0 1 2 3 4 <= Índices
lista = ["Fiap", 10, True, 34.56]
# Exibindo a lista
print(lista)
>> ['Fiap', 10, True, 34.56]
# Exibindo a terceira posição (índice 2)
print(lista[2])
>> True
# Modificando a primeira posição (índice 0)
lista[0] = "CTP"
# Exibindo a lista
print(lista)
>> ['CTP', 10, True, 34.56]
```

Legenda:

Amarelo: output da linha anterior

Demais cores: linhas de código

Pontos de observação:

Os elementos destas estruturas são delimitados com colchetes '[']':

```
lista = ["Fiap", 10, True, 34.56]
```

Os elementos são separados por vírgula:

```
"Fiap", 10, True, 34.56
```

Quando queremos especificar um elemento, colocamos o número do índice entre colchetes:

```
lista[2]
```

Manipulação de Listas

Pelas listas serem dinâmicas, conseguimos efetuar várias operações com elas através de métodos.

O que é um método?

Como uma lista é um objeto, um método nada mais é do que funções pertencentes a um objeto, no nosso caso: Lista.

Poderia fazer analogias para eu entender melhor?

Carro é um objeto	→	acelerar e brecar são métodos
Cadeira é um objeto	→	sentar , arrastar são métodos
Porta é um objeto	→	Abrir , trancar são métodos

Função:

`list()` ou `[]`

Ambos iniciam uma lista vazia.

Exemplo:

```
lista1 = list()
print(lista1)
>> []
lista2 = []
print(lista2)
>> []
```

Transcrição do exemplo:

A lista1 criada com a função list().
A lista2 criada com os colchetes.

Método

append(elemento)

Adiciona um elemento no final da Lista.

Exemplo:

```
lista = list()
lista.append(22)
lista.append("Lógica")
print(lista)
>> [22, 'Lógica']
```

Transcrição do exemplo:

Cria uma lista vazia.
Adiciona os elementos 22 e “Lógica” na lista.
Exibe o conteúdo da lista.

Exercício:

Faça um programa que crie uma lista vazia e peça para o usuário digitar elementos até digitar o caractere '.' (ponto).

Ao final exibir o conteúdo da lista.

```
Digite algo ou '.' para finalizar:  
-> Edson  
-> 51  
-> 1.71  
-> Python  
-> .  
[Edson, 51, 1.71, Python]
```

Método

`insert(posição, elemento)`

Insere um elemento numa posição (índice) específica na lista.

Exemplo:

```
lista = [22, "Lógica"]  
>> [22, 'Lógica']  
lista.insert(1, 57.7)  
>> [22, 57.7, 'Lógica']
```

Transcrição do exemplo:

Inicia uma lista com os valores 22 e “Lógica”.
Insere o elemento 57.7 no índice 1 (segunda posição) da lista.

Teste e responda:

Caso o índice colocado na posição seja superior a quantidade de elementos, o que acontece?

Método

pop([posição])

Remove um item da lista pela posição (índice). A *[posição]* é opcional, se ela for inibida, remove o último elemento, se ela for colocada, remove o índice correspondente.

Exemplo:

```
lista = [22, 57.7, "Lógica"]  
lista.pop()  
>> [22, 57.7]  
lista.pop(0)  
>> [57.7]
```

Transcrição do exemplo:

Inicia uma lista com três valores.
Remove o último elemento.
Remove o elemento com índice 0.

Teste e responda:

Caso o índice colocado entre parênteses seja diferente da quantidade de elementos, o que acontece?

Método

`remove(elemento)`

Remove um item da lista pelo elemento (conteúdo).

Exemplo:

```
lista = [22, 57.7, "Lógica"]  
lista.remove(57.7)  
>> [22, "Lógica"]
```

Transcrição do exemplo:

Inicia uma lista com três valores.
Remove o elemento 57.7.

Teste e responda:

Caso o elemento colocado entre parênteses não exista na lista, o que acontece?

Método

`index(elemento)`

Retorna o número do índice onde o elemento está na lista.

Exemplo:

```
lista = [22, 57.7, "Lógica"]  
indice = lista.index("Lógica")  
print(f"Índice = {indice}")  
>> Índice = 2
```

Transcrição do exemplo:

Inicia uma lista com três valores.
Pesquisa e retorna o índice onde está o elemento “Lógica”
Exibe o índice retornado

Teste e resposta:

Caso o elemento colocado entre parênteses não exista na lista, o que acontece?

Método

count(elemento)

Conta quantos elementos específicos há na lista

Exemplo:

```
lista = [22, 22, 57.7, "Lógica"]
qtd = lista.count(22)
print(f"Quantidade de 22 = {qtd}")
>> Quantidade de 22 = 2
print(f"Quantidade de 90 = {lista.count(90)}")
>> Quantidade de 90 = 0
```

Transcrição do exemplo:

Inicia uma lista com quatro valores.
Conta e exhibe quantos elementos 22 há na lista.
Conta e exhibe quantos elementos 90 há na lista.

Função

len(objeto)

Conta quantos elementos há em um objeto.

Exemplo:

```
lista = [22, 22, 57.7, "Lógica"]  
qtd_elementos = len(lista)  
print(f"Quantidade = {qtd_elementos}")  
>> Quantidade = 4
```

Transcrição do exemplo:

Inicia uma lista com quatro valores.
Conta quantos elementos há na lista.
Exibe a quantidade (4) de elementos.

Desafio:

somente

```
MENU
```

```
0 - SAIR  
1 - Criar / Iniciar uma lista  
2 - Inserir um elemento na lista  
3 - Inserir elementos até digitar ponto (.)  
4 - Remover elemento pelo índice  
5 - Remover elemento pelo conteúdo  
6 - Listar a lista
```

```
Escolha: _
```

Escreva este programa de uma forma que em nenhum momento haja algum erro que aborte o programa.

Dicas:

As funções e métodos: `list()`, `append()`, `pop()` e `remove()` são primárias no código, ou seja, elas agem diretamente na solução dos problemas enquanto as demais funções e métodos ajudam a validar as rotinas para que não haja erro na execução do programa.

Função

sum(lista)

Soma os elementos numéricos contidos em uma lista.

Exemplo:

```
lista = [19, 4, 25, 33, -5]
print(sum(lista))
>> 76
```

Transcrição do exemplo:

Inicia uma lista com cinco valores.
Soma os elementos da lista.
Exibe a somatória.

Teste e responda:

Caso um ou mais dos elementos não forem numéricos,
o que acontece?

Operador

+

Junta listas na ordem em que são apresentadas.

Exemplo:

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
lista3 = lista1 + lista2
print(f"Lista1 = {lista1}")
>> Lista1 = [1, 2, 3]
print(f"Lista2 = {lista2}")
>> Lista2 = [4, 5, 6]
print(f"Lista3 = {lista3}")
>> Lista3 = [1, 2, 3, 4, 5, 6]
```

Transcrição do exemplo:

Cria duas listas (lista1 e lista2) com 3 valores cada.
Em uma terceira lista concatena a lista1 com a lista2
Exibe as 3 listas.

Método

`extend(lista)`

Adiciona uma lista no final da outra.

Exemplo:

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
lista2.extend(lista1)
print(f"Lista2 = {lista2}")
>> Lista2 = [4, 5, 6, 1, 2, 3]
print(f"Lista1 = {lista1}")
>> Lista1 = [1, 2, 3]
```

Transcrição do exemplo:

Cria duas listas (lista1 e lista2) com 3 valores cada.
No final da lista2 adiciona a lista1
Exibe as duas listas.

Dúvida:

O operador + e a o método
extend() fazem a mesma
coisa ou há alguma
diferença entre eles?

Faça testes e descubra

Preencha as lacunas:

Antes de aprendermos os próximos métodos, analise o código abaixo e preencha os pontilhados com o que você acha que será impresso”

```
x = 6
y = 5
x = y
print(f"x = {x} | y = {y}")
l1 = [1, 3, 5]
l2 = [2, 4, 6]
l1 = l2
print(f"l1 = {l1} | y = {l2}")
l1.append(7)
l2.append(8)
print(f"l1 = {l1} | y = {l2}")
```

x = | y =

l1 = | l2 =

l1 = | l2 =

Teste este código e confira os resultados

Constatação:

```
x = 6
y = 5
x = y
print(f"x = {x} | y = {y}")
l1 = [1, 3, 5]
l2 = [2, 4, 6]
l1 = l2
print(f"l1 = {l1} | y = {l2}")
l1.append(7)
l2.append(8)
print(f"l1 = {l1} | y = {l2}")
```

As variáveis comuns (x e y) apresentaram os resultados das atribuições esperados. O valor de y, que era 5, foi atribuído a x, fazendo com que ambas passassem a ter o valor 5.

Quando atribuímos uma estrutura a outra (l1 = l2), não lidamos com o conteúdo da estrutura, como ocorre com variáveis simples. Nesse caso, atribuímos a referência (endereço onde a estrutura está alocada) de uma variável para a outra.

Sendo assim, qualquer modificação realizada em uma das estruturas reflete automaticamente na outra.

Como fazemos para criar uma cópia das estruturassem usar a referência?

Vejam o próximo método.

Método

`copy()`

Cria uma cópia da estrutura e atribui a outra.

Exemplo:

```
lista1 = [1, 2, 3]
lista2 = lista1.copy()
lista2.append(4)
print(f"Lista1 = {lista1}")
>> [1, 2, 3]
print(f"Lista2 = {lista2}")
>> [1, 2, 4]
```

Transcrição do exemplo:

Cria a lista1 com 3 valores.
Copia na lista2 o conteúdo da lista1.
Adiciona um elemento na lista 2.
Exibe as duas listas.

Teste e responda:

Caso a lista2 tivesse algum conteúdo, o que aconteceria com este conteúdo?

Método

`sort(reverse)`

Ordena os elementos da lista. O parâmetro `reverse=True` permite que seja ordenada em ordem decrescente.

Exemplo:

```
lista = [19, 4, 25, 33, -5]
lista.sort()
print(lista)
>> [-5, 4, 19, 25, 33]
lista.sort(reverse=True)
print(lista)
>> [33, 25, 19, 4, -5]
```

Transcrição do exemplo:

Cria a lista com valores embaralhados.
Ordena e exibe a lista em ordem crescente.
Ordena e exibe a lista em ordem decrescente.

Método

`reverse()`

Inverte os elementos da lista deixando os primeiros como últimos e os últimos como primeiros.

Exemplo:

```
lista = [19, 4, 25, 33, -5]
lista.reverse()
print(lista)
>> [-5, 33, 25, 4, 19]
```

Transcrição do exemplo:

Cria a lista com valores embaralhados.
Inverte os itens na lista.

Exercício:

Utilizando **somente** os métodos e funções explicados até o momento, faça um programa que pegue uma lista com itens distintos e crie outras duas listas, uma em ordem crescente e a outra em ordem decrescente. Apresente as três listas no final conforme o exemplo do output abaixo:

```
Lista original.....: [34, 23, -11, 45, 12]  
Lista crescente.....: [-11, 12, 23, 34, 45]  
Lista decrescente...: [45, 34, 23, 12, -11]
```

Lembre-se:

Ao final devem existir três listas diferentes. Não usar a mesma lista para ordenar.